*Proceedings of the 21st INDIACom; INDIACom-2027; IEEE Conference ID: 74162*
*2027 14th International Conference on "Computing for Sustainable Global Development", 07th–09th April, 2027*

# Simulation Throughput and Learner Scaling for Reinforcement-Learning Control of a Renewable-Powered PEM Electrolyzer

Anonymous authors
Anonymous affiliation

***Abstract*—Degradation of proton-exchange-membrane (PEM) electrolyzer stacks unfolds over years, so control strategies for renewable-powered operation must be screened in simulation, and the cost of that screening is a high-performance-computing problem. We characterise it for a one-megawatt-class plant simulated at a one-minute control interval. Process-parallel simulation on an eight-core workstation reaches $75\,800$ environment steps per second but stops scaling at the physical cores; Karp-Flatt and clock-frequency measurements point to the processor lowering its clock under load rather than to serial work. Restructuring the computation does far better: an episode-parallel simulator, which advances thousands of episodes in lockstep and reproduces the scalar simulator exactly, evaluates a $5184$-episode rule grid 295 times faster than a process pool on the same machine, and its throughput follows a Hockney overhead-plus-work model. Training of the Soft Actor-Critic (SAC) learner is bound by the learner and not by the simulator, so concurrent runs raise throughput. We also document a benchmarking pitfall: a vectorized SAC configuration documented as one update per transition performed one per 32, so steps per second overstated learning throughput about thirty-fold, and cheaper update ratios suffice only at some degradation weights. Finally, a simple hysteresis rule removes most of the apparent advantage of the learned controller, from up to 86 per cent more hydrogen at equal degradation to 1.4 and 2.6 per cent, so the baseline matters as much as the compute. No physical stack was operated.**

***Index Terms*—high-performance computing, parallel simulation, vectorization, scaling analysis, reinforcement learning, PEM water electrolysis, update-to-data ratio, performance benchmarking**

## I. Introduction

Green hydrogen from variable renewable generation depends on electrolyzer stacks that last. In proton-exchange-membrane (PEM) electrolyzers, degradation accumulates over years and depends on how the stack is operated: frequent load fluctuations during dynamic operation are reported to accelerate it [1], and the mechanisms are reviewed in [2], [3]. Testing a control strategy on hardware therefore consumes a substantial fraction of a stack's life, and simulation is the practical way to screen strategies. Its usefulness is set by two computational questions: how fast can the plant and its operating environment be simulated, and what does it cost to train a learning-based controller against that simulator? We address both for a $1\,\mathrm{MW}$-class PEM electrolyzer powered by a hybrid solar–wind resource at a one-minute control interval, where the control problem is cheap per step but needs very many steps. This is a high-performance computing (HPC) question in miniature. The contributions are measurements, one correction and one implementation technique, not a new learning algorithm:

- **Simulator scaling.** Throughput and parallel efficiency against worker count with placement controlled, and a diagnosis of the efficiency loss.
- **Episode-parallel simulation.** A batched simulator, validated to reproduce the scalar one exactly, that evaluates a rule grid two orders of magnitude faster than a process pool on the same machine, characterised with Hockney's overhead-plus-work model.
- **Learner scaling and a benchmarking pitfall.** Where Soft Actor-Critic (SAC) training time goes, how a documented one-update-per-transition configuration performed one update per 32 transitions, and why configurations should be compared by time to a stated quality.
- **A fair controller comparison.** Learned policies against grids of simple rules, including a hysteresis rule, at equal degradation.

This is a simulation study: no physical stack was operated, the degradation model is an assumed lumped model calibrated to a nominal life, and the sub-hourly resource variability is synthetic (Section VI).

## II. Related Work

Degradation of PEM water electrolysis is reviewed in [2], [3], dynamic operation in [1], and degradation models for power allocation and energy management in [4]. Deep reinforcement learning has been applied to scheduling an electric–hydrogen station with a data-driven electrolyzer model [5], at a coarser granularity than the one-minute control considered here. Engines that run many environments in parallel have been built because environment execution is often the slowest part of a reinforcement-learning system [6]; ours is a small Python model, so we study process-level parallelism and then episode-level batching. The number of gradient updates per collected transition, the update-to-data ratio (UTD), is a first-order determinant of off-policy performance and is treated as an explicit variable in [7]–[9]; comparing training speeds across configurations that differ in it compares different amounts of learning work.

## III. Workload

*Plant.* The plant has 280 cells of $1000\,\mathrm{cm}^2$, rated at $j_\mathrm{r} = 2.0\,\mathrm{A\,cm^{-2}}$ and $60\,^\circ\mathrm{C}$, with the cathode at $30\,\mathrm{bar}$. The cell voltage is the sum of a reversible term with a Nernst pressure correction, Butler–Volmer activation losses, ohmic loss, a concentration term and the degradation state $\Delta V_\mathrm{deg}$. Hydrogen follows Faraday's law derated by a Faradaic efficiency that accounts for crossover. A balance-of-plant load of $1.5\,\%$ of rated power is drawn whenever the stack is on; the rated plant power is $1049\,\mathrm{kW}$ and the plant cannot run below $36.8\,\mathrm{kW}$. The model is parameterised from typical values for this cell class and is not fitted to a device.

*Degradation.* $\Delta V_\mathrm{deg}$ is a scalar cell-voltage rise in microvolts with five additive terms per step: a base rate while on, a high-current term, a ramp term, a fixed cost per on/off transition, and an idle rate while off. The coefficients were chosen so that the threshold rule below projects a stack life of about five years. That anchor is an assumption and not a measurement, so we report *relative* comparisons only.

*Control problem.* Control is a Markov decision process with a one-minute step and a one-day episode ($T = 1440$ steps). The action $a_t \in [-1, 1]$ sets a fraction $\frac{1}{2}(a_t + 1)$ of the *currently available* renewable power, so the setpoint never exceeds the resource. The eight-dimensional observation holds the available power, its 15-minute mean, the previous setpoint, current density, normalised degradation, hydrogen rate and the time of day. The reward is $w_1 \tilde{m}_{\mathrm{H}_2} - w_2 \widehat{\Delta V_\mathrm{deg}} - w_3 |\Delta j| / j_\mathrm{r}$ with fixed scales; $w_2$ is swept to trace a yield–degradation frontier.

*Resource and baselines.* The resource is ERA5 hourly data [10] for Kutch, Gujarat (2019), converted with a PVWatts-style PV model [11] and a turbine power curve (our own implementations) and upsampled to one minute with Ornstein–Uhlenbeck noise that preserves each hourly mean. 293 days train and 72 are held out. No measured one-minute data are available to us, so the noise amplitudes are assumptions. Three simple controllers are the baselines: a *threshold rule* that follows the resource above a $10\,\%$ idle threshold and idles below it, the same rule without the threshold (a constant action of $+1$), and a tuned slew-rate-limited rule.

## IV. Methodology

*Simulator and parallelism.* One step maps the commanded power to a current density, evaluates the polarisation model and updates the degradation state; the power-to-current inversion uses a 4096-point table with two fixed-point corrections. We parallelise across independent episodes in separate processes (Section V-A) and, separately, advance many episodes in lockstep inside one process (Section V-B).

*Benchmark protocols.* Simulator workers run random-action episodes of the bare environment. Each builds its environment and warms up; all pass a barrier, and timing runs from the barrier to the last finish. *Strong scaling* fixes the work (64 episodes, 92 160 steps) and gives $S(n) = T(1)/T(n)$ and $E(n) = S(n)/n$; *weak scaling* uses 8 episodes per worker. Workers are unpinned, pinned one per physical core, or pinned onto hyper-thread pairs; bootstrap intervals over repeats describe this machine only. The learner is timed on `model.train()` over a replay buffer already filled, so environment stepping is excluded, varying device, batch size and thread count; for concurrency, $k$ independent processes start together and the aggregate rate is total updates over the span from first start to last finish. The batched simulator and the scalar environment run the same hysteresis rule on one machine, back to back.

TABLE I
Systems. Timing results are from the workstation, except the batched-simulator benchmark, which ran on the cloud CPU kernel and reports same-machine ratios. The cloud GPU notebook and an earlier server (4× NVIDIA H200, 128-core AMD EPYC 9535) trained and evaluated learned policies only and contribute no timing claim.

| | |
|---|---|
| Workstation | Intel Core i9-9900KF (8 cores, 16 threads, 3.6 GHz, `powersave` governor), 94 GiB, NVIDIA RTX A5000, Ubuntu 20.04, Python 3.12, PyTorch 2.14.1, Stable-Baselines3 2.9.0; shared with other users |
| Cloud GPU notebook | 4 vCPU Intel Xeon 2.0 GHz, 31 GiB, NVIDIA Tesla T4, PyTorch 2.11.0 |
| Cloud CPU kernel | 4 vCPU (2 cores, 2 threads each) Intel Xeon 2.2 GHz, Linux, Python 3.13, NumPy 2.1.3 |

*Update-to-data accounting.* Let $N$ be the number of vectorized environments, $F$ the `train_freq` and $G$ the `gradient_steps`. In Stable-Baselines3 [12], which provides the SAC implementation [13], [14] we use, a collection phase takes $F$ *vectorized* steps, that is $FN$ transitions, before $G$ updates, so

$$\mathrm{UTD} = \frac{G}{F\,N}. \tag{1}$$

Our earlier runs ($N = F = G = 32$) were documented as one-to-one but have $\mathrm{UTD} = 1/32$; one-to-one at $N = F = 32$ needs $G = 1024$.

*Learning experiments.* SAC uses two hidden layers of 256 units, learning rate $3 \times 10^{-4}$, batch size 512, a replay buffer of $10^6$, 32 environments and 300 000 steps per run, with $w_1 = 1$ and $w_3 = 0.1$. The discount factor $\gamma = 0.95$ was chosen from in-sample pilots on training days (0.999 was worse) and never from the held-out days. The frontier experiment trains at $w_2 \in \{1.41, 5.32, 12.9, 20\}$ with two seeds each and evaluates each policy once on the 72 held-out days. A follow-up on a cloud notebook (one job at a time) trained update ratios $1/8$ and $1/4$ at $w_2 = 20$, evaluated on training days against a rule fixed beforehand (degradation at most $1.10\times$ and hydrogen at least $0.95\times$ those of the full-ratio policies); the ratio that passed was then trained with six further seeds at $w_2 = 12.9$ and 20 and evaluated once on the held-out days. Rules are evaluated on the same days, every grid point is reported and none is tuned on them: slew limits (13 values, with and without the idle threshold), power caps (9 values), and a hysteresis rule (72 settings) that confirms a start after the resource has stayed above 0.08 or 0.15 of rated for 5 or 20 minutes and a stop

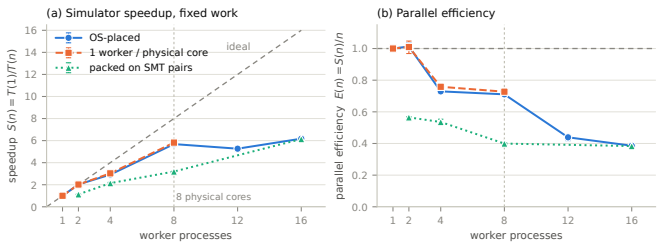



Fig. 1. Simulator speedup and parallel efficiency for fixed total work. Dashed lines: ideal. Bars: bootstrap intervals over repeats. The vertical line marks the 8 physical cores.

after 5 or 20 minutes below 0.03, with a cap of 0.5, 0.7 or 1.0 and a slew limit of 0.001, 0.003 or 0.01 per minute.

## V. Results

### A. Simulator throughput and scaling

One core simulates 12 300 steps/s, or 8.5 simulated days per second. With 8 workers the throughput is 69 800 steps/s (speedup 5.7, efficiency 0.71) and with 16 workers 75 800 steps/s (speedup 6.2); scaling flattens at the physical-core count (Fig. 1), and at the highest rate a simulated year takes 6.9 s. Placement does not matter: one worker per physical core and operating-system placement agree, two workers on one hyper-thread pair give only 1.13× the throughput of one, and 16 workers on 8 cores add 8.6 % over 8. Efficiency nevertheless falls from about 1 at two workers to about 0.75 at four, with or without pinning. Giving every worker identical episodes removes load imbalance and the drop persists (the slowest worker takes at most 1.03× as long as the fastest). The sampled core clock, however, falls from 5.0 GHz with one worker to 3.7 GHz with eight and tracks the per-worker rate (from 12 500 to 9110 steps/s); the rate per GHz changes by a factor of only 0.98. We attribute the loss to the processor lowering its clock under multi-core load (the machine runs the `powersave` governor) and not to the decomposition. This is an inference from one clock reading per worker per run on a shared machine, not a controlled experiment. As a cross-check, the Karp–Flatt serial fraction $e(n) = (1/S(n) - 1/n)/(1 - 1/n)$ is −0.013, 0.12 and 0.058 at two, four and eight workers: under Amdahl's law it would be constant, whereas here it rises and then falls, the signature of a step in per-core throughput. Weak scaling agrees, with eight workers delivering 0.70 of eight times the single-worker rate.

### B. Batching episodes instead of adding processes

Episodes never interact, so one step of $B$ episodes can be one set of array operations instead of $B$ Python-level steps. We implemented this as a batched simulator that reuses the plant and degradation models and mirrors the scalar step, including the single-precision rounding of the action and of the observation seen by the rules. Eight tests compare it with the scalar environment step by step and episode by episode on ordinary and deliberately extreme profiles (square wave, saturated plant, sparse spikes, power below the plant minimum): hydrogen, degradation and reward agree with relative difference 0 and mean efficiency to $10^{-15}$. On the 72 held-out days a random sample of 24 grid episodes agrees with maximum relative difference 0.

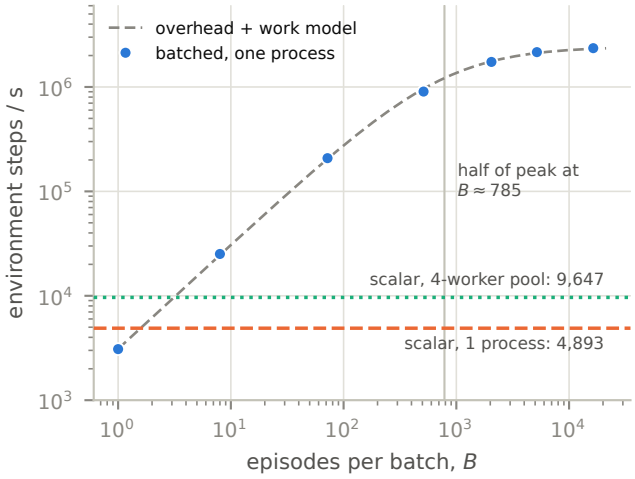



Fig. 2. Throughput of the episode-parallel simulator against batch size (one process), with the scalar environment in one process and in a four-worker pool on the same machine (cloud CPU kernel, real held-out days). Dashed curve: overhead-plus-work model fitted to the points.

On the cloud CPU kernel the scalar environment reaches 4890 steps/s in one process and 9650 in a four-worker pool (a factor of 2.0, since four vCPUs are two cores with hyper-threads). The batched simulator in one process reaches 2 350 000 steps/s at $B = 16\,384$ (Fig. 2). The time per lockstep step is well described by $t(B) = a + bB$ with a fixed overhead $a = 0.32$ ms and $b = 0.41$ µs per episode (largest fit error 6 %), which is Hockney's model: asymptotic rate 2 430 000 steps/s and half-performance length $n_{1/2} = a/b \approx 780$ episodes, below which the overhead dominates. The 72-setting hysteresis grid over the 72 held-out days is 5184 episodes: one batch evaluates it in 2.6 s and the four-worker scalar pool needs 774 s, a factor of 295 (580 against one scalar process). These are same-machine ratios; the workstation was not available when this was built. The batched simulator serves rollouts of a fixed policy (evaluation, rule grids, sweeps); SAC training remains bounded by the learner.

### C. Learner throughput

At a correct update ratio SAC is limited by the learner. The GPU performs 171 updates/s and the rate does not change when the batch grows from 256 to 2048, so the update is latency-bound and not compute-bound; the CPU performs 98 updates/s with 4 threads and 110 with 8, a GPU advantage of only 1.6× (Fig. 3). Because the rate does not depend on the number of environments, more environment parallelism does not shorten training. The remaining lever is concurrent independent runs: on the GPU one process delivers 180 updates/s, two together 255 and four 330, so

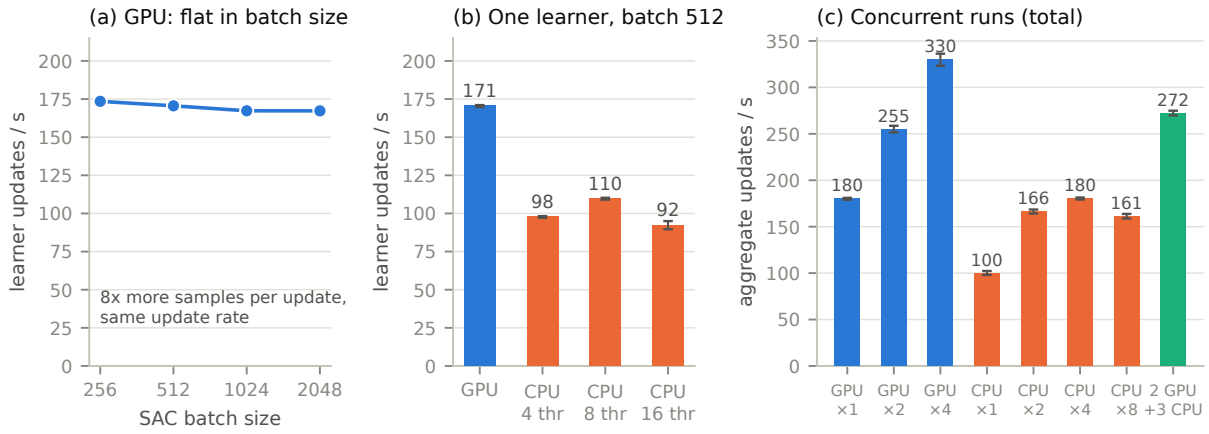



Fig. 3. SAC learner throughput (updates only; environment stepping excluded). (a) GPU rate is flat from batch 256 to 2048. (b) One learner, batch 512, by device and CPU thread count. (c) Total rate of $k$ concurrent runs.

aggregate throughput grows while each process slows down. CPU learners saturate sooner, and mixing devices is worse than the GPU alone (two GPU plus three CPU processes reach 272 updates/s), plausibly because CPU learners take cores from the GPU processes' launch threads, which we did not isolate. The best configuration (`gpu_x4`) gives 330 updates/s in total. Evaluating learned policies is also costly: rollouts that include them ran 2.8× slower than rule-based ones on the same workers, because the library's prediction call costs more per step than the plant.

### D. Update-to-data ratio and time to quality

The repository configuration performed 9060 updates in 300 000 steps, a measured ratio of 0.031 against a documented ratio of one (over $2\times10^6$ steps, about 62 500 updates instead of $\approx 2\times10^6$). An earlier benchmark reported a 26× speedup of this configuration over the library default at "identical learning dynamics" (different hardware; not reproduced here); that is the order of magnitude that removing $31/32$ of the gradient work produces, so the speedup is largely less learning per step. Throughput in steps/s should be reported together with the update-to-data ratio.

Whether a cheaper ratio suffices depends on the weight $w_2$. At $w_2 = 0.1$, SAC at ratios $1/32$ to 1 reaches the same plateau, 3.0–3.3 % below a constant-action rule on training days, at its first checkpoint, and further learning changes nothing, while the full ratio costs a factor of 20 in wall time; the plateau is a property of the problem and not of the budget. At the higher weights the cheap ratio does not learn the effect of the weight. At $w_2 = 20$ the cheap ratio gives 76.1 µV/day, close to its $w_2 = 12.9$ result, whereas the full ratio gives 50.9. Intermediate ratios (Table II) show that the effect is learned gradually, not at a threshold. Against the tolerance fixed beforehand, $1/4$ passes (degradation 1.09× and hydrogen 1.05× those of the full ratio) and $1/8$ does not (1.28×); the pass is marginal, and on held-out days the $1/4$ policies stop short of the full-ratio degradation (Section V-E). The 300k-step runs took 13 min at $1/4$ and 7.5 min at $1/8$ on the cloud notebook. An earlier sweep on the H200 server (65 runs, 13 weights, 5 seeds, $2\times10^6$ steps, ratio $1/32$, $\gamma = 0.999$) used 6.7× more steps per run but 4.8× fewer updates than the corrected runs and reached 65.6 µV/day at $w_2 = 20$ against 49.0 for the full-ratio policies; it differs in discount factor as well, so it compares configurations, and its files record no timing.

TABLE II
UPDATE RATIO AT $w_2 = 20$, $\gamma = 0.95$, 300K STEPS: MEAN OF TWO SEEDS ON EVERY FIFTH *training* DAY (IN-SAMPLE). THE LAST COLUMN IS DEGRADATION RELATIVE TO THE FULL RATIO (TOLERANCE FIXED IN ADVANCE: 1.10).

| Update ratio | Trained on | Seeds | $H_2$ [kg/d] | $\Delta V$ [µV/d] | trans. [/d] | $\Delta V$ vs. 1 [×] |
|---|---|---|---|---|---|---|
| 1/32 | workstation | 2 | 212 | 76.1 | 2.2 | 1.49 |
| 1/8 | Kaggle | 2 | 206 | 65.2 | 1.4 | 1.28 |
| 1/4 | Kaggle | 2 | 194 | 55.5 | 1.4 | 1.09 |
| 1 | workstation | 2 | 186 | 50.9 | 1.3 | — |

### E. Controller comparison on held-out days

Fig. 4 compares the policies with the non-dominated envelope of all rule settings, evaluated on the same held-out days, which favours the rules. At $w_2 \leq 5.32$ the policy lies on the envelope (−1.1 % and −0.64 %). At higher weights the one-dimensional slew-limit and power-cap rules collapse, and against them the policy appears to gain 24 % ($w_2 = 12.9$) and 86 % ($w_2 = 20$) more hydrogen at equal degradation. The 72-setting hysteresis grid does not collapse, and it cuts these to 2.7 % and 14 %. Six further seeds at ratio $1/4$ give 1.4 % at $w_2 = 12.9$ (range 0.23–2.2 %, positive for 6 of 6 seeds) and 2.6 % at $w_2 = 20$ (1.1–4.8 %, all positive): consistent in sign but small. The full-ratio policies at $w_2 = 20$ reach lower degradation (49.0 against 56.4 µV/day) with a larger advantage (14 %, two seeds), at the sparse end of the envelope; we treat that as an open question. A sensitivity sweep of the three baseline rules over 13 coefficient variants and five resource

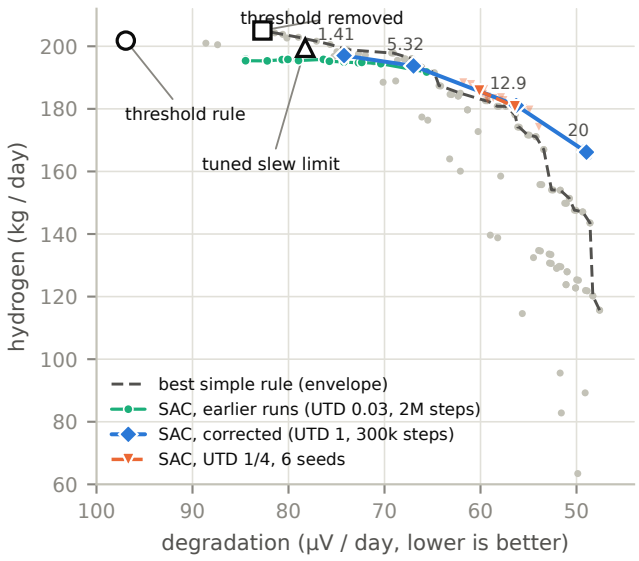



Fig. 4. Hydrogen against degradation on the 72 held-out days (lower degradation to the right). Grey: every point of the slew-limit, power-cap and hysteresis grids; dashed: their non-dominated envelope. Blue: SAC at the full ratio (mean of two seeds; faint dots are seeds), labelled with $w_2$. Orange: six further seeds at ratio $1/4$ (upper point $w_2 = 12.9$, lower $w_2 = 20$). Green: the earlier SAC runs. Open markers: the three named rules.

scenarios (84 units, 8 709 120 steps in 223 s on 6 workers) shows hydrogen yield ranking the rules identically everywhere, while projected life varies by a factor of 16 across the assumed coefficient intervals and the lifetime ranking changes in 6 of 13 variants: lifetime conclusions depend on coefficients the model cannot pin down.

## VI. Limitations

*No hardware validation:* the plant model uses typical values for the cell class and the degradation model is assumed and calibrated to a nominal life, so we claim neither absolute lifetimes nor transfer of controller rankings to hardware. *Synthetic data:* the one-minute variability is generated and all days come from one site and year of ERA5 data, so we make no claim about other sites or measured data. *Scale:* scaling and learner measurements come from one shared workstation; the intermediate-ratio and extra-seed runs were trained on a free cloud notebook; the batched-simulator timings are same-machine ratios on a four-vCPU cloud kernel; the batched simulator was not used for learned policies; we make no claim about larger systems. *Statistics:* most settings have two seeds, six more at ratio $1/4$ support the sign and rough size of the gap but not a significance test, and the pilots and update-ratio study are in-sample on training days. *Baselines:* a finer rule grid or a model-predictive controller could close more of the gap; we tried neither. The weights $w_2$ have no economic calibration.

## VII. Implications for HPC Practice

1) *Restructure the computation before adding processes.* Batching episodes gave 295 times the throughput of a process pool on the same machine, against a speedup of 5.7 on 8 workers; it pays above the half-performance length ($\approx 780$ episodes here), which should be measured for any workload.
2) *Report the update-to-data ratio and time to a stated quality, not steps per second.*
3) *Scale out runs, not environments or batch size,* because the learner is latency-bound and independent of the environment count (best aggregate 330 updates/s).
4) *Check the processor clock before blaming the code* for efficiency loss; the Karp–Flatt fraction was not constant and the clock fell with load.
5) *Do not count hyper-threads as cores* here: 16 workers on 8 cores added only 8.6 %.
6) *Spend compute on the baseline as well as the learner:* a 72-setting hysteresis grid is cheap next to training and removed most of the learned controller's apparent advantage.

## VIII. Conclusion

We characterised the computational cost of screening electrolyzer control strategies in simulation. Process-parallel simulation reaches 75 800 steps/s on 16 workers, and its inefficiency is the processor clock falling under load and not the decomposition. An exact episode-parallel simulator evaluates a 5184-episode rule grid 295 times faster than a process pool on the same machine. SAC training is bound by the learner, a documented one-to-one update ratio was in fact 0.031, and a cheaper ratio suffices at a low degradation weight but only partly at high weights, so configurations should be compared by time to a stated quality. A hysteresis rule closes most of the learned controller's low-degradation advantage (1.4 % and 2.6 % remain over six seeds), so baselines must be chosen with care. These are simulation results on an assumed degradation model and a synthetic resource; hardware validation, measured resource data, batched evaluation of learned policies and a model-predictive baseline are the necessary next steps. Code, raw results and the scripts that generate every number and figure are publicly available; the repository link is withheld for double-blind review and will be given in the camera-ready version.

## References

[1] H. Sayed-Ahmed, Á. I. Toldy, and A. Santasalo-Aarnio, "Dynamic operation of proton exchange membrane electrolyzers—critical review," *Renewable and Sustainable Energy Reviews*, vol. 189, p. 113883, 2024.
[2] Q. Feng, X.-Z. Yuan, G. Liu, B. Wei, Z. Zhang, H. Li, and H. Wang, "A review of proton exchange membrane water electrolysis on degradation mechanisms and mitigation strategies," *Journal of Power Sources*, vol. 366, pp. 33–55, 2017.
[3] A. Bernhardt, K. Lange, and K. Ilse, "Degradation mechanisms in PEM water electrolysis: Diagnosis and impact," *Advanced Materials Technologies*, 2026.
[4] A. Makhsoos, M. Kandidayeni, B. G. Pollet, and L. Boulon, "Proton exchange membrane water electrolyzers degradation models review: implications for power allocation and energy management," *Journal of Power Sources*, vol. 655, p. 238003, 2025.
[5] D. Li, L. Liu, and H. Liao, "Deep reinforcement learning-based scheduling for an electric–hydrogen integrated station using a data-driven electrolyzer model," *Applied Sciences*, vol. 16, no. 7, p. 3605, 2026.

[6] J. Weng *et al.*, "EnvPool: A highly parallel reinforcement learning environment execution engine," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022, arXiv:2206.10558.

[7] W. Fedus, P. Ramachandran, R. Agarwal, Y. Bengio, H. Larochelle, M. Rowland, and W. Dabney, "Revisiting fundamentals of experience replay," in *Proc. 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 3061–3071.

[8] X. Chen, C. Wang, Z. Zhou, and K. Ross, "Randomized ensembled double Q-learning: Learning fast without a model," in *Proc. International Conference on Learning Representations (ICLR)*, 2021, arXiv:2101.05982.

[9] P. D'Oro, M. Schwarzer, E. Nikishin, P.-L. Bacon, M. G. Bellemare, and A. Courville, "Sample-efficient reinforcement learning by breaking the replay ratio barrier," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[10] H. Hersbach *et al.*, "The ERA5 global reanalysis," *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999–2049, 2020.

[11] A. P. Dobos, "PVWatts version 5 manual," National Renewable Energy Laboratory, Tech. Rep. NREL/TP-6A20-62641, 2014.

[12] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *Proc. ICML*, 2018.

[14] T. Haarnoja *et al.*, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.